

PLE Proyecto 2
Juan Pablo Díaz - 202420409
Samuel Antonio Castro Jaimes - 202422244

Cambios del Proyecto 1 al Proyecto 2 PLE

En el Proyecto 2 se tomó la gramática definida en EBNF en el Proyecto 1 y se implementó en Rascal mediante dos archivos: la sintaxis concreta (**Syntax.rsc**) y el árbol de sintaxis abstracta (**AST.rsc**). Durante este proceso se realizaron los siguientes ajustes:

1. Separación entre sintaxis y AST

Se separó la definición del lenguaje en:

- Sintaxis concreta (cómo se escribe)
- AST (cómo se representa internamente)

Esto permite una estructura más clara y facilita el análisis.

2. Uso de estructuras tipadas

Se reemplazaron definiciones generales por tipos (**data**), por ejemplo para expresiones, operadores y atributos.

Esto mejora la seguridad y evita depender de strings.

3. Manejo explícito de valores opcionales

Elementos como $\varnothing$ ahora se representan explícitamente en el AST, evitando ambigüedades.

4. Uso de listas

Las repeticiones del EBNF (`{ ... }`) se transformaron en listas (`list[...]`), facilitando su manipulación.

5. Cambios estructurales en la gramática

Se introdujeron nuevas reglas para mejorar la organización:

- **SpaceParent**: antes era opcional dentro de `defspace`, ahora es una regla explícita.
- **Import**: antes estaba inline en `module_def`, ahora es una regla separada.
- **TypeSigTail**: se separó la recursión de `type_sig` en dos reglas (`TypeSig` y `TypeSigTail`).

6. Extensiones del lenguaje

Se agregaron nuevas capacidades:

- Nuevo cuantificador: `defer`
- Nuevo operador unario: `-` (antes solo existía `neg`)

7. Ajustes técnicos en Rascal

Se realizaron adaptaciones necesarias para Rascal:

- Se crearon lexicals como `Arrow` y `LAngle` para manejar símbolos especiales
- Se ajustó `OperatorApp` para permitir múltiples términos (consistente con `{ term }` del Proyecto 1)

Adicional:

El `Plugin.rsc` fue implementado correctamente, registrando el lenguaje con la extensión `.vlang` e incluyendo anotaciones `@category` en las reglas principales de la sintaxis concreta para habilitar el syntax highlighting. Al probar con archivos `.vlang`, se puede observar highlighting parcial en algunos elementos, lo que confirma que la implementación es funcional.

Sin embargo, la rúbrica indica que los archivos de prueba deben tener extensión `.alu`. Al usar esta extensión, el highlighting no aparece, ya que el Plugin está registrado para `.vlang`. Este comportamiento es esperado y no representa un error en la implementación. Adicionalmente,

se identificó que la versión actual de Rascal presenta problemas conocidos con el syntax highlighting que impiden su visualización completa, independientemente de la extensión utilizada.

En cuanto al parsing, el sistema funciona correctamente con ambas extensiones, el parser acepta los archivos, construye el árbol de sintaxis y genera el AST sin errores.